TASK 1:

Train.py:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
import joblib

df=pd.read_csv("weather.csv")
df=df.drop(columns=["date"])

x=df.drop(columns=["weather"])
y=df["weather"]

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)

models={
    "KNN":KNeighborsClassifier(),
    "SVM":SVC(),
    "Decision Tree":DecisionTreeClassifier()
}

best_model=None
best_accuracy=0
best_name=""

print("Model Accuracy")

for name, model in models.items():
    model.fit(x_train,y_train)
    y_pred=model.predict(x_test)
    accuracy=accuracy_score(y_test,y_pred)
```

```
    print(f"{name}: {accuracy*100}%")

    if accuracy > best_accuracy:
        best_accuracy=accuracy
        best_model=model
        best_name=name

print(f"Best Model: {best_name} ({best_accuracy*100}%)")

joblib.dump(best_model,"weather_model.pkl")
print("Model saved as weather_model.pkl")
```

test.py:

```
import tkinter as tk
import joblib

try:
    model=joblib.load("weather_model.pkl")
except:
    print("Please run train.py first")
    exit()

def predict():
        values=[
            float(prec.get()),
            float(max_temp.get()),
            float(min_temp.get()),
            float(wind.get())   ]

        prediction=model.predict([values])[0]
        result.configure(text=f"Prediction: {prediction}")

root=tk.Tk()
root.title("Weather Prediction App")
root.geometry("350x400")
```

```
tk.Label(root, text="Enter Weather Features", font=("Arial", 14, "bold italic")).pack(pady=5)

tk.Label(root,text="Precipitation").pack()
prec=tk.Entry(root)
prec.pack()

tk.Label(root, text="Max Temp").pack()
max_temp=tk.Entry(root)
max_temp.pack()

tk.Label(root, text="Min Temp").pack()
min_temp=tk.Entry(root)
min_temp.pack()

tk.Label(root, text="Wind").pack()
wind=tk.Entry(root)
wind.pack()

tk.Button(root,text="Predict",command=predict).pack(pady=10)

result=tk.Label(root,text="",font=("Arial",12))
result.pack()

root.mainloop()
```

TASK 2:

Train.py:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor
from sklearn.preprocessing import LabelEncoder
```

```
import joblib

df=pd.read_csv("car_price.csv")

encoders={}
cols=["Brand","Model","Fuel_Type","Transmission"]

for col in cols:
    le=LabelEncoder()
    df[col]=le.fit_transform(df[col])
    encoders[col]=le
print(df.head())

x=df.drop(columns=["Price"])
y=df["Price"]

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)

models={"KNN":KNeighborsRegressor(),
        "SVR":SVR(),
        "Decision Tree":DecisionTreeRegressor()
        }

best_model=None
best_score=0
best_name=""

print("Model Performance")

for name,model in models.items():
    model.fit(x_train,y_train)
    y_pred=model.predict(x_test)
    score=r2_score(y_test,y_pred)

    print(f"{name}:{score}")
```

```
    if score>best_score:
        best_score=score
        best_model=model
        best_name=name

print(f"Best Model:{best_name}({best_score})")

joblib.dump(best_model,"car_model.pkl")
joblib.dump(encoders,"encoders.pkl")

print("Model saved")
```

test.py:

```
import tkinter as tk
import joblib

try:
    model=joblib.load("car_model.pkl")
    encoders=joblib.load("encoders.pkl")
except:
    print("Run train.py first")

def predict():
    brand=encoders["Brand"].transform([b.get()])[0]
    model_name=encoders["Model"].transform([ml.get()])[0]
    year=int(yr.get())
    engine=float(eng.get())
    fuel=encoders["Fuel_Type"].transform([ft.get()])[0]
    transmission=encoders["Transmission"].transform([tn.get()])[0]
    mileage=int(mg.get())
    doors=int(dr.get())
    owner=int(own.get())

    data=[[brand,model_name,year,engine,fuel,transmission,mileage,doors,owner]]
```

```
    prediction=model.predict(data)[0]

    result.configure(text=f"Prediction:{round(prediction,2)}")
root=tk.Tk()
root.title("Car Price Prediction")
root.geometry("400x500")

tk.Label(root,text="Enter Car Details",font=("Arial",14,"bold italic")).pack(pady=5)

tk.Label(root,text="Brand").pack()
b=tk.Entry(root)
b.pack()

tk.Label(root,text="Model").pack()
ml=tk.Entry(root)
ml.pack()

tk.Label(root,text="Year").pack()
yr=tk.Entry(root)
yr.pack()

tk.Label(root,text="Engine Size").pack()
eng=tk.Entry(root)
eng.pack()

tk.Label(root,text="Fuel Type").pack()
ft=tk.Entry(root)
ft.pack()

tk.Label(root,text="Transmission").pack()
tn=tk.Entry(root)
tn.pack()

tk.Label(root,text="Mileage").pack()
mg=tk.Entry(root)
```

```
mg.pack()

tk.Label(root,text="Doors").pack()
dr=tk.Entry(root)
dr.pack()

tk.Label(root,text="Owner Count").pack()
own=tk.Entry(root)
own.pack()

tk.Button(root,text="Predict",command=predict).pack(pady=10)

result=tk.Label(root,text="",font=("Arial",12))
result.pack()

root.mainloop()
```

TASK 3:

Train.py:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
import joblib

df=pd.read_csv("diabetes.csv")

x=df.drop(columns=["Outcome"])
y=df["Outcome"]

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)

models={"KNN":KNeighborsClassifier(),
```

```
        "SVM":SVC(),
        "Decision Tree":DecisionTreeClassifier()
        }

best_model=None
best_accuracy=0
best_name=""

print("Model Accuracy")

for name,model in models.items():
    model.fit(x_train,y_train)
    y_pred=model.predict(x_test)
    accuracy=accuracy_score(y_test,y_pred)

    print(f"{name}:{accuracy*100}%")

    if accuracy>best_accuracy:
        best_accuracy=accuracy
        best_model=model
        best_name=name

print(f"Best Model:{best_name}({best_accuracy*100}%)")

joblib.dump(best_model,"diabetes_model.pkl")

print("Model saved")
```

test.py:

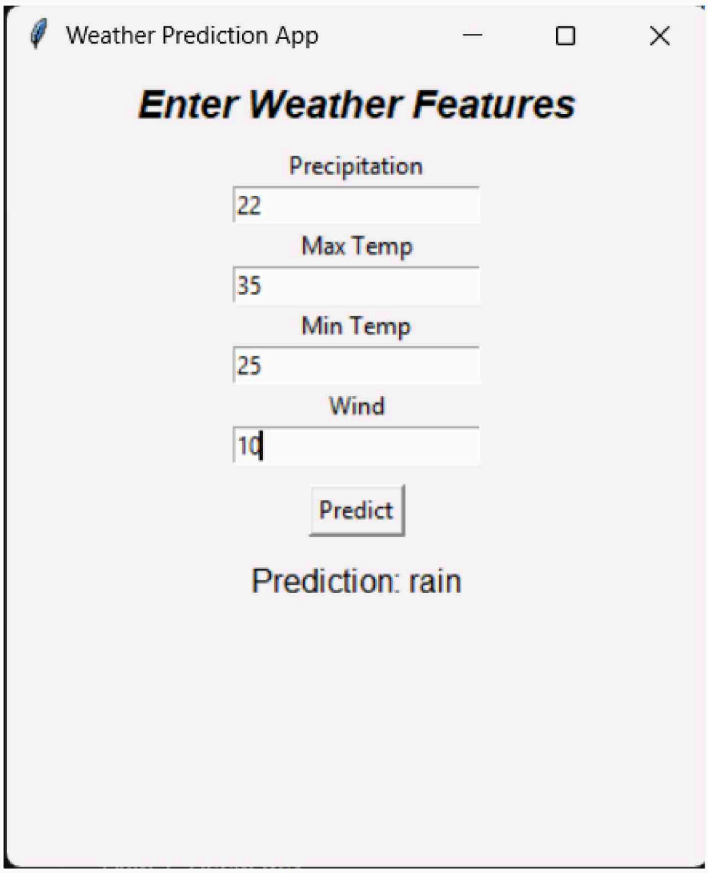
```
import tkinter as tk
import joblib

try:
    model=joblib.load("diabetes_model.pkl")
except:
```

```
    print("Run train.py first")

def predict():
        values=[
            int(prg.get()),
            int(glc.get()),
            int(bp.get()),
            int(st.get()),
            int(ins.get()),
            float(bmi.get()),
            float(dpf.get()),
            int(age.get())
        ]

        prediction=model.predict([values])[0]

        if prediction==1:
            result.configure(text="Diabetic")
        else:
            result.configure(text="Not Diabetic")

root=tk.Tk()
root.title("Diabetes Prediction")
root.geometry("400x500")

tk.Label(root,text="Enter Patient Details",font=("Arial",14,"bold italic")).pack(pady=5)

tk.Label(root,text="Pregnancies").pack()
prg=tk.Entry(root)
prg.pack()

tk.Label(root,text="Glucose").pack()
glc=tk.Entry(root)
glc.pack()
```

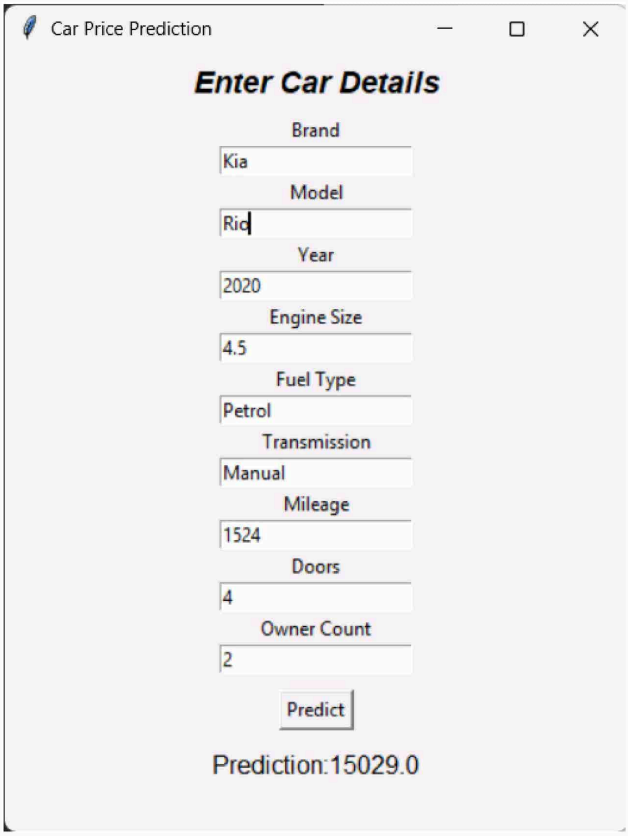
```
tk.Label(root,text="Blood Pressure").pack()
bp=tk.Entry(root)
bp.pack()

tk.Label(root,text="Skin Thickness").pack()
st=tk.Entry(root)
st.pack()

tk.Label(root,text="Insulin").pack()
ins=tk.Entry(root)
ins.pack()

tk.Label(root,text="BMI").pack()
bmi=tk.Entry(root)
bmi.pack()
tk.Label(root,text="Diabetes Pedigree Function").pack()
dpf=tk.Entry(root)
dpf.pack()

tk.Label(root,text="Age").pack()
age=tk.Entry(root)
age.pack()

tk.Button(root,text="Predict",command=predict).pack(pady=10)

result=tk.Label(root,text="",font=("Arial",12))
result.pack()

root.mainloop()
```

Outputs:

```
PS D:\Internship\Day45> python wtrain.py
Model Accuracy
KNN: 78.15699658703072%
SVM: 76.45051194539249%
Decision Tree: 74.06143344709898%
Best Model: KNN (78.15699658703072%)
Model saved as weather_model.pkl
```




```
PS D:\Internship\Day45> python ctrain.py
   Brand  Model  Year  Engine_Size  Fuel_Type  Transmission  Mileage  Doors  Owner_Count  Price
0      6     24  2020          4.2          0             1   289944      3            5   8501
1      2     19  2012          2.0          2             0     5356      2            3  12092
2      7     16  2020          4.2          0             0   231440      4            2  11171
3      0     22  2023          2.0          1             1   160971      2            1  11780
4      9     17  2003          2.6          2             2   286618      3            3   2867
Model Performance
KNN:0.1575267403370686
SVR:0.17236755901644985
Decision Tree:0.9325411580901698
Best Model:Decision Tree(0.9325411580901698)
Model saved
```




```
PS D:\Internship\Day45> python dtrain.py
Model Accuracy:

Logistic Regression: 74.68%
Decision Tree: 75.32%
Random Forest: 74.03%

Best Model: Decision Tree (75.32%)
Model saved as diabetes_model.pkl
```